

Unit III

Abstraction, Packages and Exception Handling

- 3.1 Final Keyword, Abstract class & Abstract Methods
- 3.2 Interfaces
- 3.3 System Packages and User defined Packages
- 3.4 Try, catch block and finally clause
- 3.5 User defined exceptions

3.1 final Keyword in Java – Final variable, Method and Class

The final keyword is a non-access modifier in Java. Modifiers in Java are keywords that change the behavior of classes, methods, or variables. They are two main types of modifier –

1. **Access Modifiers** - These control **who can access** a class, method, or variable.
Examples: public, private, protected and *default* (no modifier)

2. **Non-Access Modifiers** - These do not affect accessibility but instead add extra properties or behaviors.
Examples: final, static, abstract, etc.

It can be applied to variables, methods, and classes — and its meaning changes slightly in each case.

final variable

In Java, the **final keyword** is a **non-access modifier** that is used to declare **constants** (variables whose value cannot be changed once assigned).

Characteristics:

1. A **final variable** can be assigned **only once**.
2. After initialization, its value **cannot be changed**.
3. It is often used for declaring **constants** (static final).
4. If a final variable is not initialized at the time of declaration, it must be initialized in:
 - **Constructor**, or
 - **Instance initialization block**

Example 1: Simple Final Variable

```
class Demo {  
    public static void main(String[] args) {  
        final int x = 10; // final variable  
        System.out.println(x);  
    }  
}
```

Example 2: Final Variable in Constructor

```
class Student {  
    final int rollNo; // not initialized here  
    Student(int r) {  
        rollNo = r; // initialized in constructor  
    }  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
    }  
    public static void main(String[] args) {  
        Student s1 = new Student(101);  
        Student s2 = new Student(102);  
        s1.display();  
        s2.display();  
    }  
}
```

final method

In Java, a final method is declared using the **final** keyword before the method definition

Characteristics:

1. Declared as:

```
final returnType methodName() { ... }
```

2. A subclass can **inherit** the method but **cannot override (change)** its implementation.
3. It is useful when we want to keep a method's behavior **unchanged** for all subclasses.
4. But a final method can still be **overloaded** in the same class.

```
class Vehicle {  
    final void engineType() {  
        System.out.println("Engine type: Diesel"); // fixed  
    }  
  
    void wheels() {  
        System.out.println("Number of wheels: 4"); // can be overridden  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    void wheels() {  
        System.out.println("Number of wheels: 4 (Car)");  
    }  
}
```

```

    }

    public class Demo {
        public static void main(String[] args) {
            Car car = new Car();
            car.engineType(); // calls final method from Vehicle
            car.wheels();    // overridden method
        }
    }

```

final class

In Java, a **final class** is a class that **cannot be extended (inherited)**. It is declared using the **final keyword** before the class definition.

Example 1 –

```

final class Animal { // final class
    void sound() {
        System.out.println("Animals make sound");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Animal();
        a.sound();
    }
}

```

Example 2 –

```

final class Constants {
    static final double PI = 3.14159;
    static final double E = 2.71828;

    static void display() {
        System.out.println("Math Constants: PI = " + PI + ", E = " + E);
    }
}

public class Demo {
    public static void main(String[] args) {
        Constants.display();
    }
}

```

Abstract class & Abstract Method

Before learning the Java abstract class, let's understand the abstraction in Java first.

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user. Another way, it shows only essential things to the user and hides the internal details. For example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

In Java, abstract class and abstract methods help in achieving **abstraction**.

Abstract Class

An **abstract class** in Java is a class that is declared with the keyword `abstract`. It is used to **represent an incomplete class** that cannot be instantiated directly.

Key Features

1. Declared using the keyword `abstract`.
2. Cannot be **instantiated** (no objects can be created).
3. May contain **abstract methods** (without implementation).
4. May also contain **normal methods** with implementation.
5. Used for **inheritance** → subclasses must provide implementation for abstract methods.

Example 1: Abstract Class with Abstract Method

```
abstract class Animal {
    abstract void sound(); // abstract method (no body)

    void sleep() {          // concrete method
        System.out.println("Sleeping...");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        // Animal a = new Animal(); // ✗ Not allowed
        Animal a = new Dog();        // ✔ Allowed (upcasting)
        a.sound();
        a.sleep();
    }
}
```

Abstract Method in Java

An **abstract method** in Java is a method that is **declared without implementation** (no method body). It is meant to be **overridden** in a subclass.

Syntax –

```
abstract returnType methodName(parameters); //no method body and abstract
abstract void printStatus();
```

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike{
    abstract void run();
}
class Honda4 extends Bike{
    void run(){System.out.println("running safely");}
    public static void main(String args[]){
        Bike obj = new Honda4();
        obj.run();
    }
}
```

Output

running safely

Another example of abstract class and abstract method

```
abstract class Shape{
    abstract void draw();
}
class Rectangle extends Shape{
    void draw(){System.out.println("drawing rectangle");}
}
class Circle1 extends Shape{
    void draw(){System.out.println("drawing circle");}
}
class TestAbstraction1{
    public static void main(String args[]){
        Shape s=new Circle1();
        s.draw();
    } }
}
```

Output

drawing circle

```
abstract class Shape{
    abstract void draw();
}

class Rectangle extends Shape{
    void draw(){System.out.println("drawing rectangle");}
}

class Circle1 extends Shape{
    void draw(){System.out.println("drawing circle");}
}

class TestAbstraction1{
    public static void main(String args[]){
        Shape s=new Rectangle ();
        s.draw();
    } }
```

Output

drawing rectangle

3.2 Interfaces in Java

An interface in Java is a blueprint of a class. It has static constants and abstract methods. The interface in Java is a mechanism to achieve abstraction. There can be only abstract methods in the Java interface, not method body. It is used to achieve abstraction and multiple inheritances in Java.

How to declare an interface?

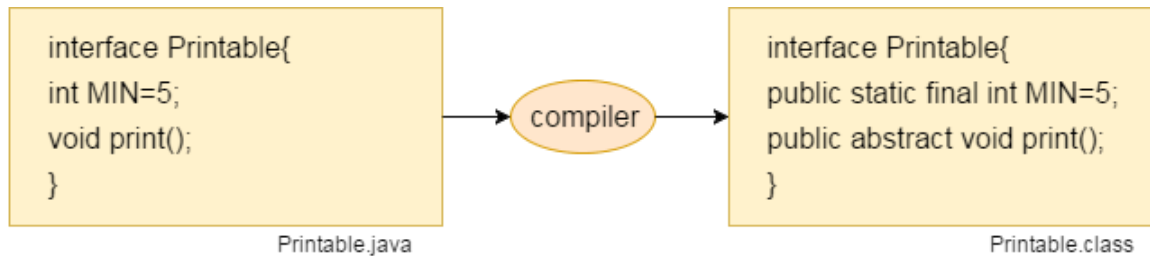
An interface is declared by using the **interface** keyword. It provides total abstraction; means all the methods in an interface are declared with the empty body, and all the fields are public, static and final by default. A class that implements an interface must implement all the methods declared in the interface.

Syntax:

```
Interface <interface_name> {
    // declare constant fields
```

```
// declare methods }
```

The Java compiler adds public and abstract keywords before the interface method. Moreover, it adds public, static and final keywords before data members.



Java Interface Example

In this example, the printable interface has only one method, and its implementation is provided in the A6 class.

```
interface printable{  
    void print();  
}  
  
class AB implements printable{  
    public void print(){ System.out.println("Hello");}  
    public static void main(String args[]){  
        AB obj = new AB();  
        obj.print();  
    } }  
}
```

Output
Hello

Java Interface Example: Drawable

In this example, the Drawable interface has only one method. Its implementation is provided by Rectangle and Circle classes. In a real scenario, an interface is defined by someone else, but its implementation is provided by different implementation providers. Moreover, it is used by someone else. The implementation part is hidden by the user who uses the interface.

```
//Interface example1 - Single inheritance  
interface Drawable{  
    void draw();  
}
```

```

class Rectangle implements Drawable{
public void draw(){System.out.println("drawing rectangle");}
}
class Circle implements Drawable{
public void draw(){System.out.println("drawing circle");}
}
//Using interface: by third user
class TestInterface1{
public static void main(String args[]){
Circle C=new Circle();
C.draw();
}}

```

Output

drawing circle

//Interface example2 – Multiple inheritance

```

interface Drawable1{
void draw1();
}

interface Drawable2{
void draw2();
}

class Rectangle implements Drawable1,Drawable2{
public void draw1(){System.out.println("drawing rectangle");}
public void draw2(){System.out.println("drawing circle");}
}

class TestInterface2{
public static void main(String args[]){
Rectangle R=new Rectangle();
R.draw1();
R.draw2();
}
}

```

Output –

D:\java>javac TestInterface2.java

D:\java>java TestInterface2
drawing rectangle
drawing circle

3.3 System Packages and User defined Packages

Packages

A package in Java is a way to group related classes, interfaces, and sub-packages together.

Package in java can be categorized in two types :

1) **Built-in packages** → Already provided by Java.

- java.util (Scanner, ArrayList, etc.)
- java.io (File handling classes)
- java.sql (Database classes)

2) **User-defined packages** → Created by programmers to organize their own code.

Advantage of Java Package

1. **Organized Code** → Like folders in a computer.
2. **Avoids Naming Conflicts** → Two classes with the same name can exist in different packages.
3. **Reusability** → Classes once written can be reused in other projects.
4. **Access Protection** → Packages work with access modifiers (public, protected, default, private).

Syntax & example for creating a package

```
package myPackage; // declare package at top

public class MyClass {
    public void display() {
        System.out.println("Hello from MyClass in myPackage");
    }
}
```

Save this file as MyClass.java.

To Compile: `javac -d . MyClass.java`

To Run: `java myPackage.MyClass`

Output – Hello from MyClass in myPackage

Example to create simple package

The package keyword is used to create a package in java.

//save as Simple.java

```
package mypack;

public class Simple{

    public static void main(String args[]){
```

```
        System.out.println("Welcome to package");
    }
}
```

How to compile java package

If you are not using any IDE, you need to follow the syntax given below:

```
D:/>javac -d directory javafilename
```

For example

```
D:/>javac -d . Simple.java
```

The -d switch specifies the destination where to put the generated class file. You can use any directory name like /home (in case of Linux), d:/abc (in case of windows) etc. If you want to keep the package within the same directory, you can use . (dot).

How to run java package program

You need to use fully qualified name e.g. mypack.Simple etc to run the class.

```
To Compile:   javac -d . Simple.java
```

```
To Run:       java mypack.Simple
```

```
Output –     Welcome to package
```

How to access package from another package?

There are two ways to access the package from outside the package.

1. `import package.*;`
2. `import package.classname;`

1) Using packagename.*

If you use `package.*` then all the classes and interfaces of this package will be accessible but not sub packages. The `import` keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the `packagename.*`

```
//save by package1.java
package pack;
public class package1{
    public void msg(){System.out.println("Hello");}
```

```

    }
    //save by package2.java
    package mypack;
    import pack.*;
    class package2{
    public static void main(String args[]){
    package1 obj = new package1 ();
    obj.msg();
    }
    }

```

Output:

D:\java>javac -d . package2.java

D:\java>java mypack.package2

Hello

2) Using packagename.classname

If you import package.classname then only declared class of this package will be accessible.

Example of package by import package.classname

```

    //save by A.java
    package pack;
    public class A{
    public void msg(){System.out.println("Hello");}
    }

    //save by B.java
    package mypack;
    import pack.A;
    class B{
    public static void main(String args[]){
    A obj = new A();
    obj.msg();
    }
    }

```

Output: Hello

3.4 Try, catch block and finally clause

The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

What is Exception Handling?

In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime. Exception Handling is a mechanism that handles runtime errors such as ClassNotFoundException, IOException, SQLException, RemoteException, etc

Advantage of Exception Handling

The core advantage of exception handling is to maintain the normal flow of the application. Let's take a scenario:

```
Statement 1;  
Statement 2;  
Statement 3; //exception occurs  
Statement 4;  
Statement 5
```

Suppose there are 5 statements in your program and there occurs an exception at statement 3. The statements 4,5 will not be executed due to the exception. But if we perform exception handling, the rest of the statement will be executed. That is why we use exception handling in Java.

Java try-catch block

Java try block

Java try block is used to enclose the code that might throw an exception. It must be used within the method.

Java try block must be followed by either catch or finally block.

Syntax of Java try-catch

```
try{  
  
    //code that may throw an exception  
  
}catch(Exception_class_Name ref){  
  
}
```

Syntax of try-finally block

```
try{  
  
    //code that may throw an exception  
  
}finally{  
  
}
```

Java catch block

Java catch block is used to handle the Exception by declaring the type of exception within the parameter. The declared exception must be the parent class exception (i.e., Exception) or the generated exception type. However, the good approach is to declare the generated type of exception.

The catch block must be used after the try block only. You can use multiple catch block with a single try block.

Problem without exception handling

Let's try to understand the problem if we don't use a try-catch block.

```
public class TryCatchExample1 {  
  
    public static void main(String[] args) {  
  
        int data=50/0; //may throw exception  
  
        System.out.println ("rest of the code");  
  
    }    }
```

Output: Exception in thread "main" java.lang.ArithmeticException: / by zero

As displayed in the above example, the rest of the code is not executed (in such case, the rest of the code statement is not printed).

There can be 100 lines of code after exception. So all the code after exception will not be executed.

Solution by exception handling

Let's see the solution of the above problem by a java try-catch block.

Example 2-

```
public class TryCatchExample2 {  
    public static void main(String[] args) {  
        try  
        {  
            int data=50/0; //may throw exception  
        }  
        //handling the exception  
        catch(ArithmeticException e)  
        {  
            System.out.println(e);  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Output:

```
java.lang.ArithmeticException: / by zero  
rest of the code
```

Java multiple catch exceptions

Java Multi-catch block

A try block can be followed by one or more catch blocks. Each catch block must contain a different exception handler. So, if you have to perform different tasks at the occurrence of different exceptions, use java multi-catch block.

Points to remember

- At a time only one exception occurs and at a time only one catch block is executed.
- All catch blocks must be ordered from most specific to most general, i.e. catch for ArithmeticException must come before catch for Exception.

Example 1

Let's see a simple example of java multi-catch block.

```
public class MultipleCatchBlock1 {  
    public static void main(String[] args) {  
        try{  
            int a[]=new int[5];  
            a[5]=30;  
        }  
        catch(ArithmeticException e)  
        {  
            System.out.println ("Arithmetic Exception occurs");  
        }  
        catch(ArrayIndexOutOfBoundsException e)  
        {  
            System.out.println("ArrayIndexOutOfBoundsException Exception occurs");  
        }  
        catch(Exception e)  
        {  
            System.out.println("Parent Exception occurs");  
        }  
        System.out.println("rest of the code");  
    }  
}
```

Output:

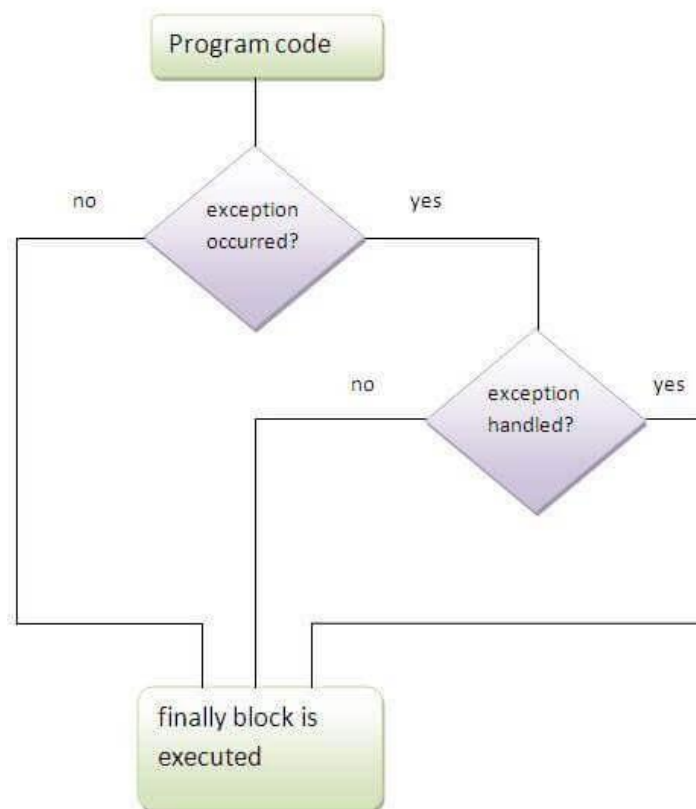
```
System.out.println("ArrayIndexOutOfBoundsException Exception occurs");  
Rest of the code
```

Java finally block

Java finally block is a block that is used *to execute important code* such as closing connection, stream etc.

Java finally block is always executed whether exception is handled or not.

Java finally block follows try or catch block.



Case 1

Let's see the java finally example where **exception doesn't occur**.

```
class TestFinallyBlock{
    public static void main(String args[]){
        try{
            int data=25/5;
            System.out.println(data);
        }
        catch(NullPointerException e){System.out.println(e);}
        finally{System.out.println("finally block is always executed");}
        System.out.println("rest of the code...");
    } }
```

Output:5

```
    finally block is always executed
    rest of the code...
```
